

Pseudo-SO Multiprogramado: Implementação de um Sistema Operacional Simulado

Lucas Resende Silveira Reis
1801444221

Rafael Hamú
202006448

João Victor Pereira Vieira
211036114

Abstract—Este trabalho descreve a implementação de um pseudo-Sistema Operacional multiprogramado, desenvolvido para a disciplina de Fundamentos de Sistemas Operacionais da Universidade de Brasília. O sistema simula os principais subsistemas de um SO real: gerenciamento de processos com escalonamento por múltiplas filas e feedback, gerência de memória virtual por paginação com algoritmo LRU local, alocação exclusiva de recursos de E/S e um sistema de arquivos com alocação contígua via first-fit.

Index Terms—Sistema Operacional, Gerenciador de Processos, Gerenciador de Memória, Gerenciador de E/S, Gerenciador de Arquivos, Escalonamento, Paginação LRU

I. INTRODUÇÃO

Em um sistema operacional multiprogramado são esperadas diversas funções que garantam transparência para facilitar o uso de recursos para o usuário. Neste trabalho, com apoio de módulos de gerenciamento de processos, memória, E/S e sistema de arquivos, foi implementado um pseudo-SO que aplica os conceitos estudados na disciplina ao longo do semestre.

O sistema lê três arquivos de entrada: `processes.txt`, com as informações dos processos a serem criados; `files.txt`, com o estado inicial do disco e as operações a serem realizadas; e `string.txt`, com as strings de referência de páginas de cada processo. A partir dessas entradas, o despachante coordena a execução completa da simulação.

II. DESCRIÇÃO DAS FERRAMENTAS E LINGUAGENS USADAS

O trabalho foi implementado em Python 3.10+, utilizando apenas a biblioteca padrão da linguagem, sem dependências externas. A escolha se justifica pela legibilidade do código, pelo domínio da linguagem por todos os integrantes do grupo e pela adequação da linguagem ao problema — que exige lógica de filas, manipulação de listas e leitura de arquivos, tarefas em que Python é direto e eficiente. Para manipulação de arquivos e caminhos, foi utilizado o módulo nativo `os`. As filas foram implementadas com `collections.deque`, estrutura da biblioteca padrão otimizada para inserção e remoção em ambas as extremidades.

III. DESCRIÇÃO TEÓRICA E PRÁTICA DA SOLUÇÃO

Nesta seção é apresentada a descrição teórica e prática das soluções que utilizamos para implementação de cada módulo.

A. Estrutura das Filas e Escalonamento

O escalonamento é realizado por meio de um relógio lógico de ticks que avança de forma sequencial e determinística, eliminando a necessidade de threads reais ou sincronização por semáforos. Todos os processos entram em uma fila global de espera ao chegar no sistema. Um processo só transita para o estado *Pronto* — e entra nas filas de escalonamento — quando consegue alocar simultaneamente memória suficiente e todos os recursos de E/S que necessita. Enquanto qualquer um dos dois faltar, o processo permanece na fila global aguardando.

O sistema mantém dois tipos de fila. A fila de tempo real implementa FIFO sem preempção: processos com prioridade 0 têm precedência absoluta e são executados até o fim antes de qualquer processo de usuário. As três filas de usuário (prioridades 1, 2 e 3) implementam múltiplas filas com feedback: a cada quantum de 1ms em que um processo não termina, ele é rebaixado para uma fila de menor prioridade. Para evitar *starvation*, um mecanismo de aging promove processos que aguardam por mais de 5 ticks consecutivos, subindo-os uma fila de prioridade.

Processos com pedidos impossíveis — working set maior que o limite da área ou recursos de E/S em quantidade superior ao total do sistema — são rejeitados na chegada com mensagem explicativa e nunca entram nas filas. Eles também são excluídos da contagem de faltas de página ao final.

B. Estrutura de Memória

O gerenciador de memória virtual por paginação gerencia um total fixo de 20 frames de 1 KiB cada. O sistema aplica um particionamento rígido na memória principal, reservando 8 frames exclusivos para processos de Tempo Real e 12 frames para processos de Usuário, garantindo que as duas áreas nunca se misturem. A alocação é efetuada no método `alocar_processo` com base no limite máximo do working set declarado pelo processo.

A substituição de páginas adota o algoritmo LRU local, implementado no método `referenciar_pagina` por meio de listas dinâmicas mapeadas por PID. A pré-carga carrega a primeira página referenciada por cada processo diretamente, sem contabilizar falta de página. Em caso de hit, a página é movida para o final do vetor, atualizando sua recência de uso. Em caso de page fault com working set cheio, o método executa descarte local removendo a página no início da lista (a menos recentemente utilizada) via `pop(0)` e insere a nova página no final. A string de referência de páginas

é percorrida integralmente durante a execução do processo, independente do tempo de processador — as duas dimensões são independentes conforme a especificação. Ao término do processo, o método `liberar_processo` devolve os frames ao pool da área correta. O histórico de faltas é armazenado em dicionário persistente para o relatório final.

C. Estrutura dos Recursos Disponíveis

O gerenciador de E/S controla 1 scanner, 2 impressoras, 1 modem e 2 dispositivos SATA. Todos os recursos são alocados de forma exclusiva e sem preempção: uma vez adquirido, o processo retém o recurso até terminar. Processos de tempo real não utilizam recursos de E/S, conforme a especificação.

A alocação é atômica (tudo-ou-nada): o processo só adquire os recursos quando consegue todos de uma vez. Se qualquer recurso estiver indisponível, nenhum é alocado e o processo permanece na fila global. Esse comportamento evita deadlock por construção, pois nenhum processo retém recurso parcial enquanto aguarda o restante. Como consequência, todos os processos válidos eventualmente conseguem executar.

D. Estrutura do Sistema de Arquivos

No sistema de arquivos, se notou necessária a adoção de algum método para monitoramento de blocos livres. Devido ao sistema ser de alocação contígua first-fit, foi notável a vantagem do uso de contagem, onde em vez de termos uma lista de n endereços livres, procuramos o primeiro bloco livre e o número n de blocos contíguos livres a partir dele. Para isso foi criado o método `blocos_livres`, que faz essa varredura e atualiza a lista de espaços livres contíguos do disco. A partir daí, o método de criar um arquivo (`_criar`) conseguia rapidamente verificar se tamanho do segmento pode ser comportado pelo disco. Ainda na função de criar arquivos, foi implementada a associação de um processo a seus determinados arquivos, o que foi utilizado no método `_deletar`, que deleta um arquivo existente no disco, caso o processo seja dono do arquivo, ou de tempo real. Por fim, foram criados métodos de seleção de operação (0 criar, 1 Deletar), chamado de `executar_operacao` e o método `imprimir_mapa`, que imprime a ocupação do disco naquele momento.

Para simular o funcionamento ideal da tabela de blocos livres contíguos, uma alteração teve de ser feita. Antes a função que retorna os espaços contíguos livres era chamada toda vez que um arquivo novo tentava ser criado. A solução encontrada foi de criar uma lista de espaços livres, verificar seu estado no início e depois somente a cada nova alteração do disco (após criar ou deletar um arquivo). Assim, o funcionamento continuou o mesmo, mas reflete mais ao comportamento de um SO.

Outra alteração foi em relação ao momento de deletar um arquivo, onde antes, a lista inteira era percorrida já que o laço só procurava qual bloco tinha o nome do arquivo a ser deletado, e após a mudança do laço, os blocos deletados são os do início do segmento até o início + tamanho. Uma outra implicação disso, foi a possibilidade de terem dois arquivos de processos diferentes com o mesmo nome, já que antes

ao deletar um arquivo, todos de mesmo nome também eram deletados, até mesmo os de outro processo.

IV. DIFICULDADES DE IMPLEMENTAÇÃO

Durante a implementação, foram encontradas dificuldades tanto em aspectos práticos do desenvolvimento quanto em pontos conceituais que, embora tenham sido abordados em sala de aula, exigiram atenção redobrada na hora de aplicá-los.

No escalonamento, um conceito que parecia trivial — o estado *Pronto* — gerou retrabalho. A ideia de que um processo só está pronto quando possui memória e recursos de E/S simultaneamente é ensinada na teoria, mas na prática a alocação dos dois em momentos separados causou inconsistências que só foram percebidas durante os testes. O mesmo vale para a relação entre operações de arquivo e tempo de processador: é natural que cada operação consuma tempo de CPU, mas isso não estava explícito na implementação inicial.

O tratamento de entradas inválidas — valores negativos e pedidos impossíveis de recursos — também não estava previsto inicialmente, pois a especificação assumia entradas bem formadas. Com os testes, ficou clara a necessidade de validação na chegada do processo, afinal não sabemos certamente quais erros o usuário que faz os testes pode vir a propor, então sendo sempre recomendável ter esse tipo de verificação.

Por fim, a contagem de faltas de página gerou uma pequena dúvida em relação ao exemplo da especificação: o gabarito indicava $P0=6$, enquanto nossa implementação produziu $P0=7$, diferença de 1 decorrente da interpretação do momento exato em que a pré-carga é aplicada. O valor de $P1=14$ foi reproduzido com exatidão.

Durante o desenvolvimento do gerenciador de memória (`memoria.py`), a principal dificuldade técnica residuiu na reconciliação de dois requisitos contraditórios da especificação: o isolamento estrito de recursos locais por processo e a persistência global de dados estatísticos.

O sistema exige que a substituição de páginas pelo algoritmo LRU ocorra de forma local, impedindo que um processo interfira nos frames alocados para outro. No entanto, ao final da simulação de todo o sistema operacional, o programa deve exibir um relatório consolidado com o total de falhas de página (page faults) de cada processo. A dificuldade inicial era que, ao encerrar o ciclo de vida de um processo e liberar seus recursos da memória principal, todo o seu estado e referências eram apagados, o que causava a perda dos dados acumulados de falhas.

No sistema de arquivos, as duas dificuldades iniciais foram a busca eficiente por blocos livres contíguos e a associação de arquivos aos seus processos criadores. A lógica de first-fit é simples conceitualmente, mas exigiu cuidado na implementação da varredura e na atualização da tabela de blocos após cada operação.

V. SOLUÇÕES ADOTADAS

Para o estado *Pronto*, foi implementada a função `_tentar_tornar_pronto` no escalonador, que tenta alo-

car memória e recursos de E/S em sequência. Se a memória for alocada mas os recursos falharem, a memória é imediatamente devolvida e o processo retorna à fila global. Somente quando ambos são garantidos o processo é admitido nas filas de escalonamento.

Para o controle de operações por tempo de processador, o dispatcher mantém um dicionário de créditos por PID. Processos rejeitados iniciam com 0 créditos. A cada operação executada, 1 crédito é consumido. Quando os créditos se esgotam, qualquer operação subsequente daquele PID imprime a mensagem de processo inexistente.

Para entradas inválidas, o método `tem_valores_invalidos` na classe `Processo` verifica campos negativos ou nulos antes de qualquer tentativa de alocação. Pedidos de E/S impossíveis são verificados pelo método `pedido_e_possivel` do gerenciador de E/S. Processos inválidos são marcados como REJEITADO e excluídos das faltas de página.

Para o sistema de arquivos, foi adotado o sistema de contagem de blocos livres contíguos descrito na seção anterior, e a tabela é atualizada somente após cada modificação no disco.

Para o gerenciamento de memória virtual, foi adotada uma estrutura híbrida de dados no construtor da classe `Memoria` para conciliar o isolamento de escopo e a persistência de dados. Enquanto as tabelas de controle ativo de frames e as filas de recência do algoritmo LRU Local são tratadas como estruturas voláteis e destruídas imediatamente pela função `liberarprocesso` para liberar a RAM virtual, o histórico de falhas de página de cada PID é armazenado de forma isolada em um dicionário persistente. Desse modo, o sistema garante a integridade e o isolamento entre os processos durante a execução, permitindo simultaneamente que os dados estatísticos de page faults sobrevivam ao término do ciclo de vida do processo para a emissão correta do relatório consolidado global.

VI. FUNÇÃO DE CADA ALUNO NA REALIZAÇÃO DO TRABALHO

- **Lucas Resende Silveira Reis** — Sistema de arquivos: implementação do first-fit, operações de criação e deleção, controle de permissões e impressão do mapa do disco.
- **Rafael Hamú Campos** — Gerência de memória: paginação com LRU local, pré-carga e contagem de page faults. Gerência de E/S: alocação atômica de recursos.
- **João Victor Pereira Vieira** — Núcleo do sistema: estrutura de filas, escalonador com feedback e aging, fila global com estado Pronto, integração dos módulos no dispatcher e testes.

VII. USO DE INTELIGÊNCIA ARTIFICIAL

Durante o desenvolvimento do trabalho, foram utilizadas ferramentas como Claude (Anthropic) como ferramenta de apoio ao aprendizado. O uso se restringiu a esclarecer dúvidas sobre a lógica de funcionamento de um sistema operacional, entender como estruturas da linguagem Python funcionam na

prática — como o deque da biblioteca `collections` e a organização de módulos — e criar casos de teste para validar o comportamento do sistema. Todo o código foi escrito e compreendido pelos próprios integrantes do grupo; a IA foi utilizada como ferramenta de estudo, não como geradora de código.

VIII. REPOSITÓRIO

O código-fonte completo do projeto, incluindo os arquivos de entrada de exemplo e os casos de teste, está disponível em:

<https://github.com/The-Jonas/Pseudo-SO-Multiprogrammed>

REFERENCES

- [1] Stallings, W. *Operating Systems: Internals and Design Principles*. Pearson Education, 2009.
- [2] Martin, R. C. *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall, 2008.
- [3] Araújo, A. P. F. *Slides da disciplina de Fundamentos de Sistemas Operacionais*. Universidade de Brasília, 2026. Disponível no Aprender 3.